



Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

LIFO (Last In, First Out)

ELE3515 - Grupo 01 - Problema 04 - Projeto

Líder:	Samuel de Lira Moraes
Redator:	Rinaldo Tavares da Silva Filho
Debatedor:	Christian Cauã Batista da Silva

Natal, 4 de novembro de 2025

Resumo

Este trabalho apresenta o projeto e a implementação de uma estrutura LIFO (*Last In, First Out*) utilizando a metodologia de projeto RTL (*Register Transfer Level*). O objetivo principal foi desenvolver um circuito digital capaz de armazenar e recuperar dados seguindo o comportamento típico de uma pilha, em que o último elemento inserido é o primeiro a ser retirado.

O projeto foi dividido em duas partes principais: o bloco da LIFO, responsável pelo controle e armazenamento dos dados, e o circuito de conversão da saída binária de 13 bits para o formato BCD, possibilitando a exibição dos valores em displays de sete segmentos. O desenvolvimento contemplou o projeto da máquina de estados, o bloco de controle, o `datapath`, e o banco de registradores, garantindo uma operação sincronizada e eficiente.

A implementação demonstrou que é possível simplificar a arquitetura da LIFO utilizando apenas um contador para gerenciar as operações de leitura e escrita, reduzindo a complexidade lógica e o número de componentes necessários. Os resultados obtidos confirmam o correto funcionamento da estrutura, validando o comportamento de empilhamento e desempilhamento característico do sistema LIFO.

Palavras-chave: LIFO; Sistemas Digitais; Circuitos Combinacionais; Projeto RTL.

Sumário

1	Fundamentação Teórica	4
1.1	LIFO	4
1.1.1	Princípio de Funcionamento	4
1.1.2	Verificação de Estados da Pilha	5
1.2	Banco de registradores	6
2	Desenvolvimento da Solução	8
2.1	Projeto da LIFO	8
2.1.1	Maquina de estado	8
2.1.2	Projeto RTL	10
2.1.3	Banco de registradores	12
2.1.4	Datapath	13
2.1.5	Bloco de controle	14
2.1.6	Circuito da LIFO	15
2.2	Projeto da Solução	16
2.2.1	Montagem Final	17
3	Conclusão	18
A	Relato semanal	20
A.1	Equipe	20
A.2	Defina o problema	20
A.3	Pontos-chaves	20
A.4	Questões de pesquisa	20
A.5	Planejamento da pesquisa	20
B	Anexos	21

1 Fundamentação Teórica

1.1 LIFO

LIFO (Last In, First Out) é uma estrutura de armazenamento utilizada quando se deseja um comportamento de pilha. Nesse tipo de estrutura, o último dado armazenado é o primeiro a ser lido, enquanto o primeiro dado gravado será o último a ser recuperado.

Esse comportamento é amplamente utilizado em sistemas digitais, especialmente em processadores, onde a pilha é usada para armazenar endereços de retorno de sub-rotinas, variáveis temporárias e dados intermediários durante a execução de operações. O princípio de funcionamento da LIFO é simples: todas as inserções e remoções ocorrem em uma única extremidade, conhecida como topo da pilha.

A pilha é uma das estruturas fundamentais da computação digital e pode ser implementada tanto por meio de software (como em linguagens de programação) quanto em hardware, utilizando registradores ou blocos de memória. Em sistemas digitais, a LIFO é comumente implementada com registradores e lógica combinacional, permitindo o armazenamento e recuperação de dados com alta velocidade.

1.1.1 Princípio de Funcionamento

A LIFO difere de outras estruturas de armazenamento por realizar suas operações sempre no mesmo ponto. Cada nova escrita adiciona um dado no topo da pilha, e cada leitura remove o dado mais recentemente inserido. Dessa forma, a ordem de leitura é sempre inversa à de gravação — o último elemento inserido será o primeiro a ser recuperado.

Essa lógica é análoga à manipulação de uma pilha de objetos físicos: ao empilhar livros sobre uma mesa, o último colocado no topo é o primeiro a ser retirado. Em termos de hardware, esse comportamento é obtido por meio de um contador e um banco de registradores que armazenam os valores.

Enquanto em uma FIFO o controle de escrita e leitura é feito com dois contadores independentes (um para o início e outro para o fim da fila) ([Vahid 2008](#)), na LIFO ambos os acessos ocorrem sobre a mesma posição de memória, sendo o índice de leitura sempre estando um unidade subtraída do índice de escreto. Portanto, é possível implementar o circuito utilizando apenas um contador, que é incrementado durante a escrita e decrementado durante a leitura, conforme ilustrado na Fig. 1.1.

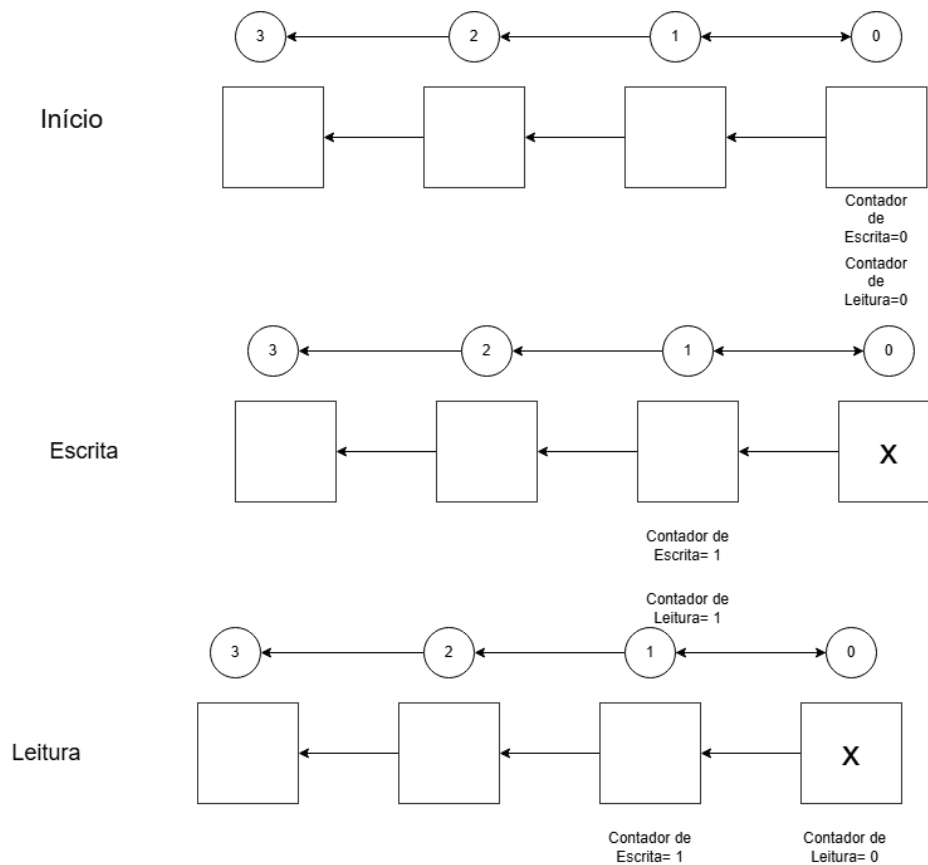


Figura 1.1: Diagrama de funcionamento do contador da LIFO

Essa otimização reduz a complexidade do hardware e o número de componentes lógicos necessários, sem alterar o comportamento da pilha.

1.1.2 Verificação de Estados da Pilha

Para evitar erros de operação, como tentar ler um dado de uma pilha vazia ou escrever em uma pilha cheia, é necessário um mecanismo de detecção de estado. Isso é feito comparando o valor atual do contador com limites predefinidos — geralmente o valor zero (pilha vazia) e o valor máximo (pilha cheia).

Na implementação com apenas um contador, um comparador com zero é usado para identificar quando a pilha está vazia. Esse comparador pode ser construído conectando todas as saídas do contador (por exemplo, 4 bits) a portas OR e aplicando um inversor na saída. Assim, o sinal de “pilha vazia” será ativado apenas quando todos os bits do contador forem zero.

Já o estado de “pilha cheia” pode ser detectado quando o contador atinge seu valor máximo permitido. Para diferenciar entre as condições de cheia e vazia, o circuito de controle também leva em consideração a última operação executada — leitura ou escrita —, seguindo o mesmo princípio usado em estruturas de fila ([Vahid 2008](#)).

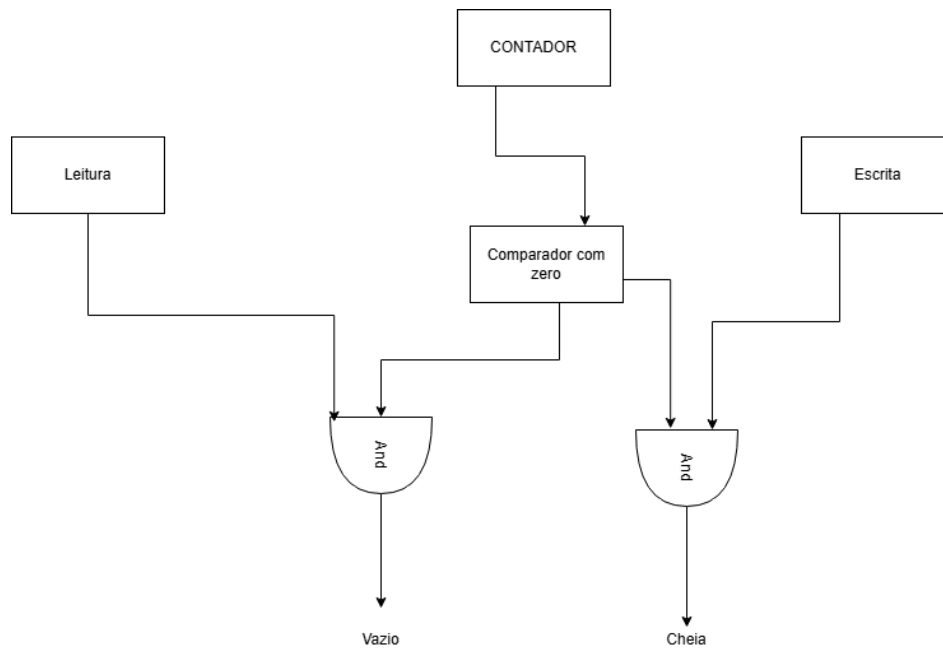


Figura 1.2: Lógica para detecção dos estados “cheia” e “vazia” na pilha LIFO

1.2 Banco de registradores

O banco de registradores é um componente essencial para o armazenamento temporário e manipulação de dados. Ele pode ser entendido como uma memória, geralmente de pequeno porte, composta por um conjunto de registradores individuais, acessados por meio de endereços. Cada registrador é capaz de armazenar um dado binário de N bits, e o banco completo contém M registradores, formando uma estrutura lógica $M \times N$, conforme Fig. 1.3.

De acordo com (Vahid 2008), um banco de registradores funciona de maneira análoga a uma memória RAM (Random Access Memory), já que ambas permitem leitura e escrita individual em cada palavra a partir de um endereço. A principal diferença está na escala e implementação: enquanto as RAMs são empregadas em armazenamentos maiores e mais compactos, os bancos de registradores são menores e costumam ser implementados com flip-flops dedicados, oferecendo acesso rápido e múltiplas portas de leitura e escrita, mas tendo tamanho físico bem maior.

Quando o sinal de escrita é ativado, o dado presente na entrada é armazenado no registrador indicado pelo endereço. De forma análoga, durante a leitura, o valor contido no registrador selecionado é amostrado na saída de dados. Esse mecanismo de endereçamento direto é o que torna o acesso “aleatório”, pois qualquer registrador pode ser acessado independentemente do anterior.

No projeto da LIFO, o banco de registradores representa a estrutura de armazenamento da pilha. O contador de endereços é responsável por indicar qual posição deve ser escrita ou lida, de acordo com as operações de leitura e escrita. Assim, o endereçamento sequencial do contador permite o correto gerenciamento dos dados no topo da pilha, garantindo o comportamento característico da LIFO.

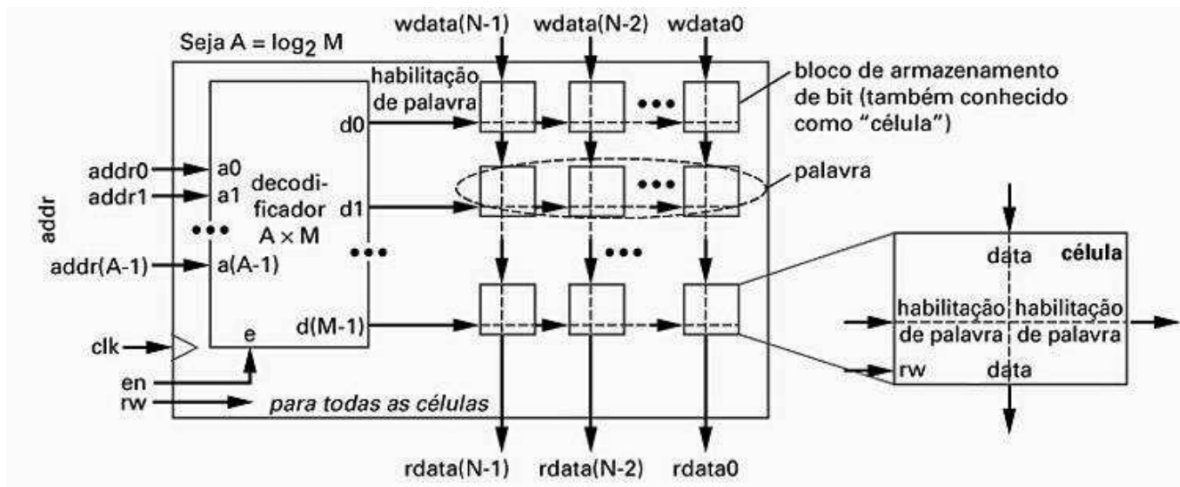


Figura 1.3: Funcionamento do Banco de registradores. Fonte: (Vahid 2008)

2 Desenvolvimento da Solução

Para simplificar o projeto da LIFO, decidiu-se dividi-lo em duas partes: a primeira parte é responsável por toda a lógica operacional da LIFO, enquanto a segunda parte é responsável por converter a saída binária de 13 bits da LIFO em quatro números no formato BCD (Codificação binária decimal), que são, em seguida, exibidos em displays de sete segmentos.

2.1 Projeto da LIFO

2.1.1 Máquina de estado

Alto nível

A máquina de estado de alto nível desenvolvida no projeto conta com 4 estados, sendo eles Inicial (00), Gravar (01), Ler (10) e Clear(11). Em cada estado tem diferentes funções sendo elas:

No estado Inicial o circuito não deve realizar nenhuma operação sobre os registradores do contador e sobre os registradores da fila, mas deverá gerar um sinal HIGH na saída fu quando o contador for igual a zero e o último estado for 01, similarmente deverá gerar um sinal HIGH na saída em quando o contador for igual a zero e o último estado for 10.

No estado de Gravar deverá gravar no o valor de `w_data` no índice do banco de registrador equivalente ao contador, e simultaneamente realizar uma soma de uma unidade no contador.

No estado de Ler deverá ler o valor do índice do banco de registradores equivalente ao contador menos 1, esse valor deve ser amostrado em `r_data`, e simultaneamente realizar uma subtração de uma unidade ao contador.

No estado de Clear deverá ser zerado todos os índices do banco de registradores e o contador.

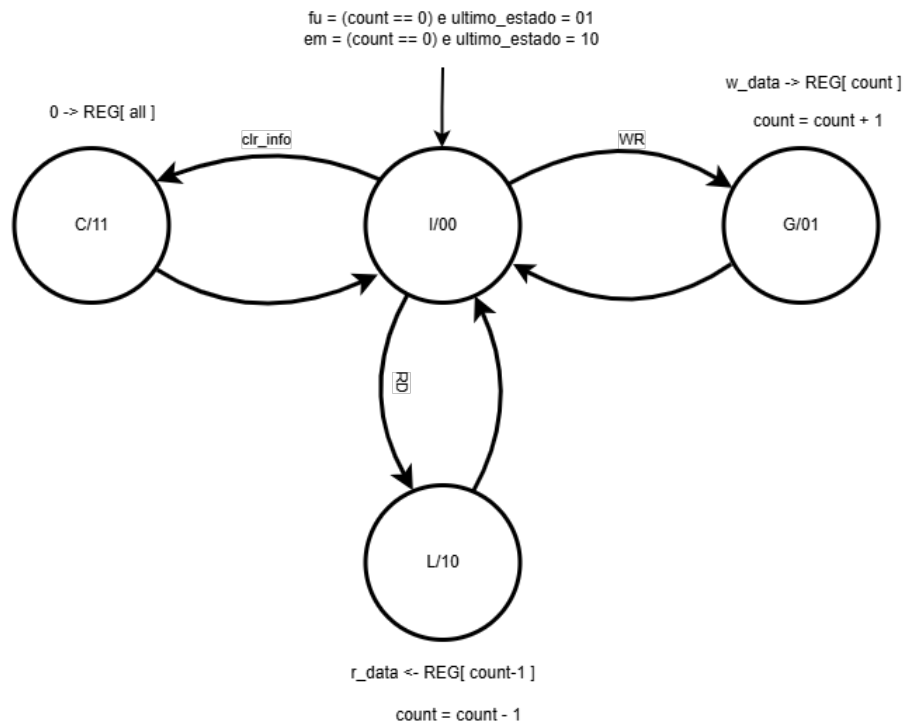


Figura 2.1: Máquina de estado de alto nível

Baixo nível

A máquina de estado de baixo nível representa a implementação detalhada do controle descrito na máquina de alto nível. Nela são especificados os sinais de controle, variáveis internas e condições lógicas que determinam as transições entre os estados.

Essa máquina trabalha diretamente com os sinais do projeto RTL (Register Transfer Level), definindo quando cada operação deve ocorrer, bem como os valores de ativação dos sinais de leitura, escrita e limpeza.

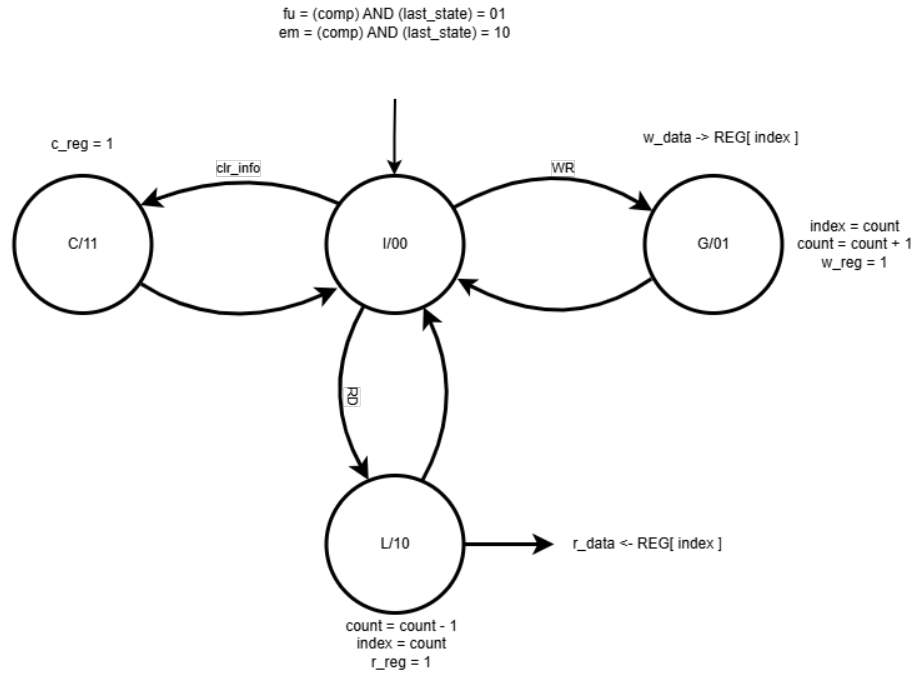


Figura 2.2: Máquina de estado de baixo nível

2.1.2 Projeto RTL

O projeto RTL conta com o **datapath**, responsável pelas operações combinacionais necessárias, como subtração, soma e comparação com zero. Além disso, possui os elementos de memória: o registrador de 4 bits do contador e os 16 registradores de 13 bits cada, que compõem a fila. O bloco de controle, por sua vez, é responsável por gerenciar os sinais que controlam o **datapath**. A representação do projeto RTL pode ser observada na Fig. 2.3.

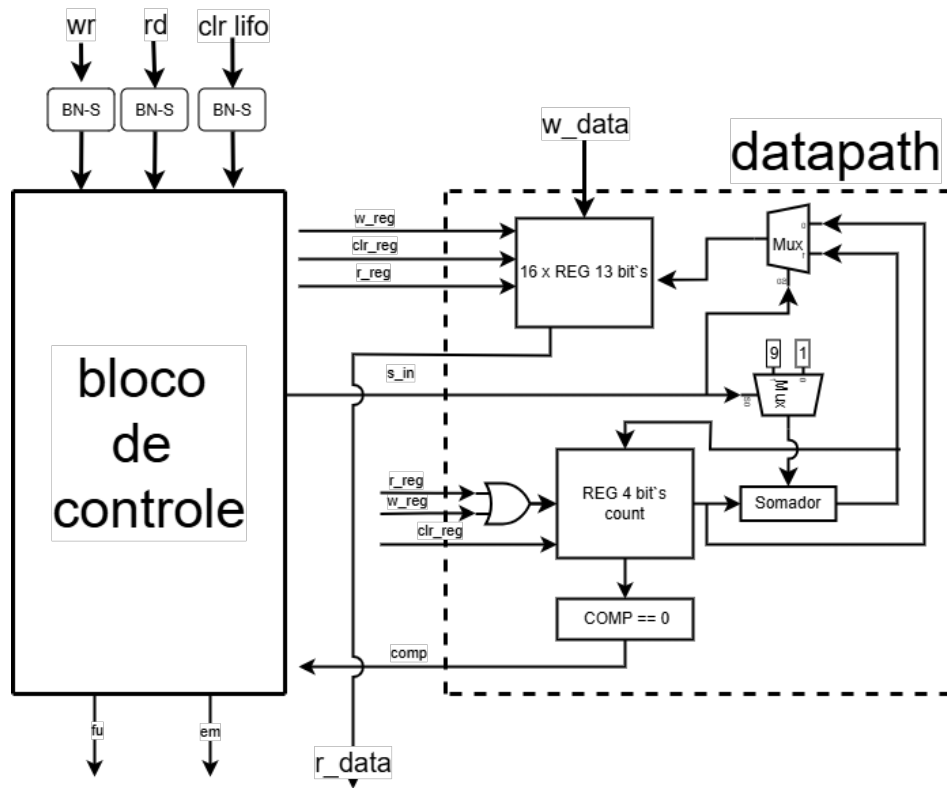


Figura 2.3: Projeto RTL LIFO

Datapath

O bloco **datapath** se comunica com o bloco de controle por meio de cinco sinais: **w_reg**, **clr_reg**, **r_reg**, **s_in** e **comp**. Os sinais **w_reg**, **clr_reg**, **r_reg** e **s_in** são gerados pelo bloco de controle e definem como o **datapath** se comporta em cada estado. Já o sinal **comp** é o **feedback** do **datapath**, indicando quando o contador está em zero, ou seja, quando a fila está cheia ou vazia.

Alguns elementos foram implementados para reduzir a quantidade de componentes combinacionais, como o multiplexador do somador, que substitui a necessidade de um segundo somador. Além disso, foi implementada uma operação lógica de OU no sinal de habilitação (**enable**) do contador, com o objetivo de simplificar a tabela-verdade do bloco de controle.

O **datapath** também recebe diretamente o sinal **w_data**, gerado pelo usuário. Esse bloco, por sua vez, fornece ao usuário apenas o sinal de saída **r_data**.

Bloco de controle

O bloco de controle recebe os sinais provenientes do **datapath** e os sinais **wr**, **rd** e **clr_lifo**, gerados diretamente pelo usuário ao operar a LIFO. Esse bloco é responsável por gerar os sinais que controlam o **datapath** e também por identificar quando a fila está cheia ou vazia, ativando as saídas **fu** e **rd** com nível lógico alto (HIGH), respectivamente.

A lógica de controle é dada pela tabela verdade Tab. 2.1.

Tabela 2.1: Tabela da Verdade logica de controle

$\overline{l_state}$	wr	rd	$\overline{clr_lifo}$	$comp$	$state$	S_in	$\overline{clr_reg}$	$\overline{w_reg}$	$\overline{r_reg}$	fu	em
0	0	0	0	x	0	0	0	0	0	0	0
0	0	x	x	1	x	1	1	0	0	0	0
0	0	0	1	0	x	1	0	1	0	0	0
0	0	1	x	0	x	0	1	0	1	0	0
0	1	x	x	x	0	0	0	0	0	0	0
0	1	x	x	x	1	0	0	0	0	1	0
1	0	x	x	x	0	0	0	0	0	0	0
1	x	x	x	x	1	0	0	0	0	0	1
1	1	x	x	x	0	0	0	0	0	0	1

A equações logicas decorrentes dessa tabela verdade são:

$$\begin{aligned}
state_1 &= \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{clr_lifo} + \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{wr} \cdot rd \\
state_0 &= \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{clr_lifo} + \overline{l_state1} \cdot \overline{l_state0} \cdot wr \\
s_in &= \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{wr} \cdot rd \cdot \overline{clr_lifo} \\
clr_reg &= \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{clr_lifo} \\
w_reg &= \overline{l_state1} \cdot \overline{l_state0} \cdot wr \cdot \overline{clr_lifo} \\
r_reg &= \overline{l_state1} \cdot \overline{l_state0} \cdot \overline{wr} \cdot rd \cdot \overline{clr_lifo} \\
em &= l_state1 \cdot comp + l_state1 \cdot l_state0
\end{aligned}$$

2.1.3 Banco de registradores

O circuito do banco de registradores implementado é composto por 16 registradores, com flip flops tipo d, de 13 bits e um decodificador de 4 bits. Cada combinação dos 4 bits de entrada do decodificador aciona o sinal de habilitação (**enable**) de um registrador diferente. Por questões de organização, optou-se por utilizar a combinação binária equivalente ao valor decimal do índice do registrador. Por exemplo, a combinação 0000 aciona o registrador 0, enquanto a combinação 1000 aciona o registrador 8, e assim por diante. A tabela-verdade do decodificador está disponível nos anexos Tab. B.2.

O circuito do banco de registradores, ilustrado na Fig. 2.4, possui como entradas os sinais **WRITE**, **READ**, **clear**, **w_data**, **count** e **ck**. Quando a entrada **WRITE** está em nível lógico alto (HIGH), o valor presente em **w_data** é gravado no registrador de índice correspondente a **count**. Esse comportamento ocorre porque o sinal **WRITE** habilita o decodificador, que, por sua vez, ativa o registrador selecionado.

A entrada **READ**, quando em nível lógico alto (HIGH), define que a saída **c_reg** será equivalente ao valor armazenado no registrador de índice definido por **count**. Esse funcionamento é obtido por meio de dois multiplexadores: o primeiro com 4 bits de seleção e 13 bits de dados, e o segundo com 1 bit de seleção e 13 bits de dados. O primeiro multiplexador é responsável por selecionar qual saída será amostrada, de acordo com o valor de **count**, de forma similar ao decodificador. O segundo multiplexador garante que o valor do registrador selecionado só seja apresentado na saída quando **READ** estiver em nível lógico alto.

A entrada **clear** está conectada diretamente ao sinal de **reset** assíncrono de todos os registradores, garantindo a reinicialização simultânea de todo o banco de registradores. Por fim, a entrada **w_data** corresponde ao valor fornecido pelo usuário, proveniente das chaves SW[12:0].

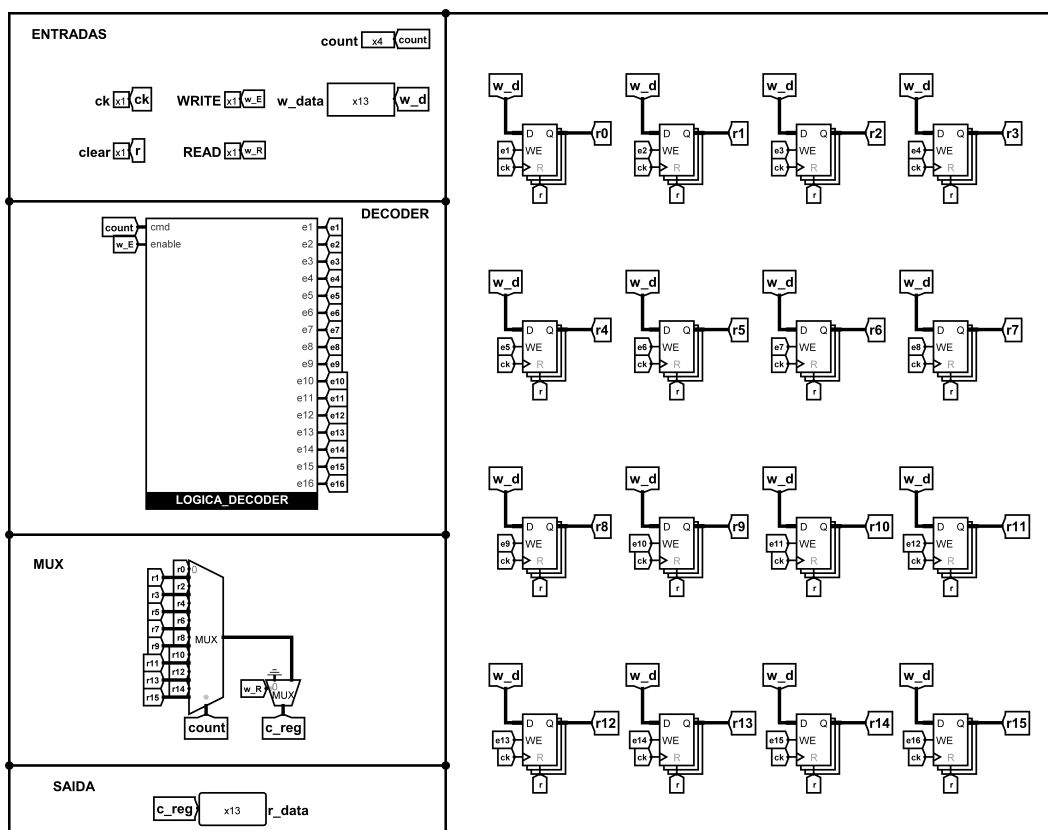


Figura 2.4: Circuito do banco de registradores

2.1.4 Datapath

O circuito desenvolvido para o datapath conta com um somador de 4 bits, um registrador de 4 bits, com flip flops tipo d, um comparador com zero de 4 bits, um banco de registradores e dois multiplexadores com 1 bit de seleção e 4 bits de dados. As entradas, conforme definido no projeto RTL, são: `w_reg`, `r_reg`, `clr_reg`, `s_in`, `w_data` e o sinal de clock.

Como o projeto da LIFO utiliza apenas um contador, é necessário realizar diferentes operações sobre ele de acordo com o estado atual: somar 1 quando estiver no estado de escrita e subtrair 1 quando estiver no estado de leitura. Para isso, foi utilizado um multiplexador de modo que, ao receber o sinal `s_in` em nível lógico baixo (LOW), seja adicionado o valor 0001, e, ao receber nível lógico alto (HIGH), seja adicionado o valor 1111, equivalente à subtração de 1.

Além disso, é importante garantir que o registrador de 4 bits que armazena o valor do contador só seja alterado (incrementado ou decrementado) nos estados de leitura ou escrita. Para isso, foi utilizada uma porta lógica OU no sinal de habilitação (`enable`) do registrador do contador. Essa porta recebe como entradas os sinais `r_reg` e `w_reg`, que são ativados apenas quando o sistema está nos estágios de leitura e gravação, respectivamente.

Para garantir que o valor correto do índice seja enviado ao bloco de registradores, foi implementado um segundo multiplexador. Quando `s_in` está em nível lógico baixo (LOW), é enviado ao bloco de registradores o valor atual do contador. Quando `s_in`

está em nível lógico alto (HIGH), é enviado o valor do contador já processado pelo somador. Dessa forma, no estado de gravação é utilizado o valor atual do contador, enquanto no estado de leitura é utilizado o valor do contador decrementado em uma unidade.

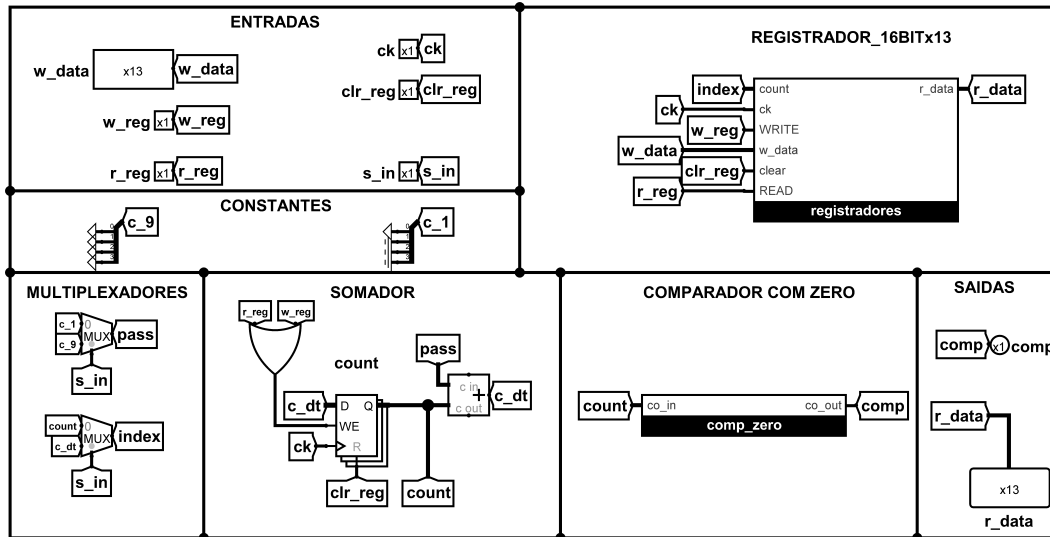


Figura 2.5: Circuito do datapath

O comparador com zero é implementado por meio de uma operação lógica OU entre todos os bits invertidos do contador, conforme ilustrado na Fig. 2.6.

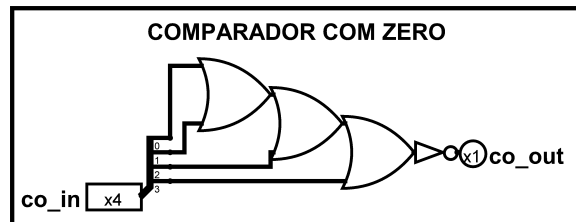


Figura 2.6: Circuito do comparador com zero

2.1.5 Bloco de controle

O circuito do bloco de controle é composto por uma lógica combinacional, descrita pela tabela-verdade apresentada na Tab. 2.1, e por um registrador de 2 bits responsável por armazenar o estado atual, que será utilizado no próximo ciclo de clock. O circuito completo pode ser observado na Fig. 2.7.

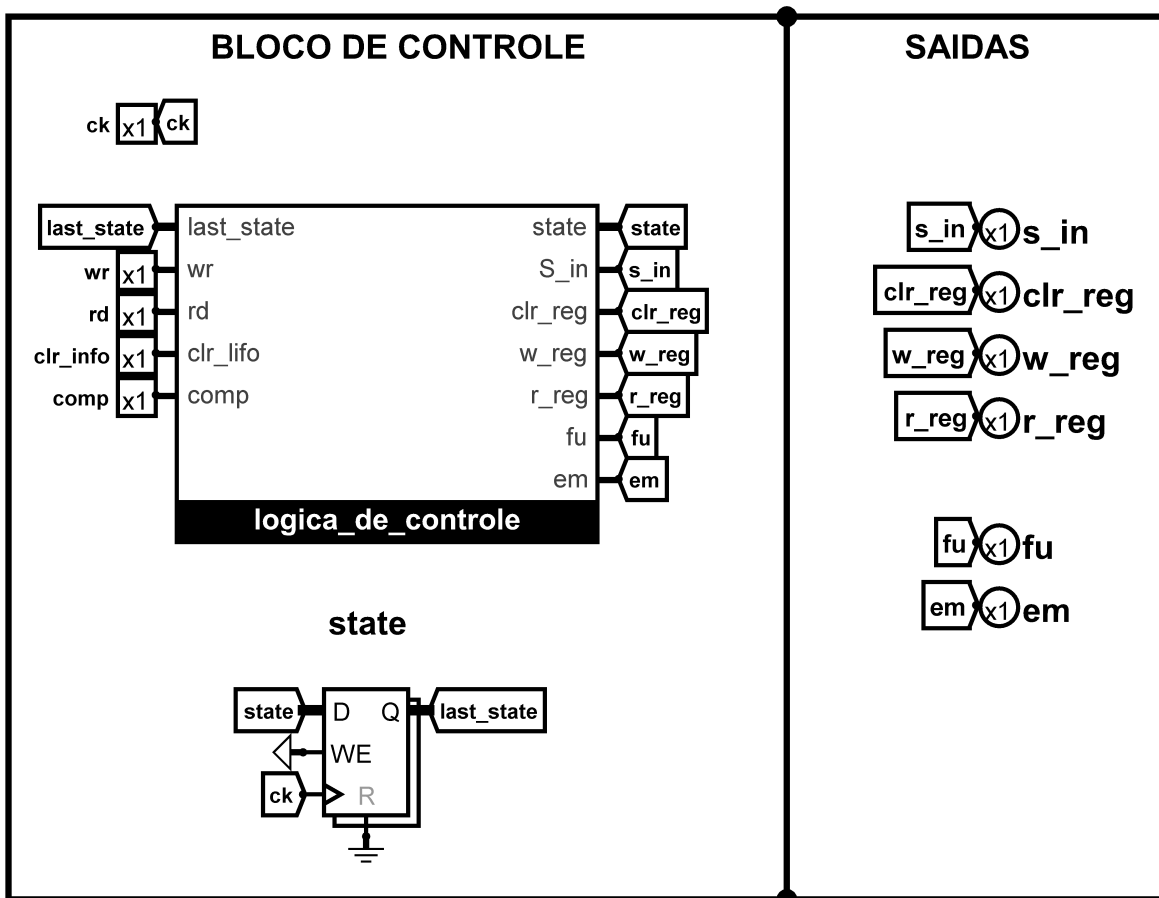


Figura 2.7: Circuito do bloco de controle

2.1.6 Circuito da LIFO

O circuito da LIFO é formado pela integração do bloco de controle com o datapath. Suas entradas são os sinais `clr_lifo`, `wr`, `rd` e `w_data`, fornecidos pelo usuário. Como saídas, o circuito gera os sinais `r_data`, `fu` e `em`, conforme mostrado na Fig. 2.8. O sinal `r_data` é posteriormente enviado a um circuito secundário responsável por converter o valor binário em formato numérico para exibição em um display de sete segmentos.

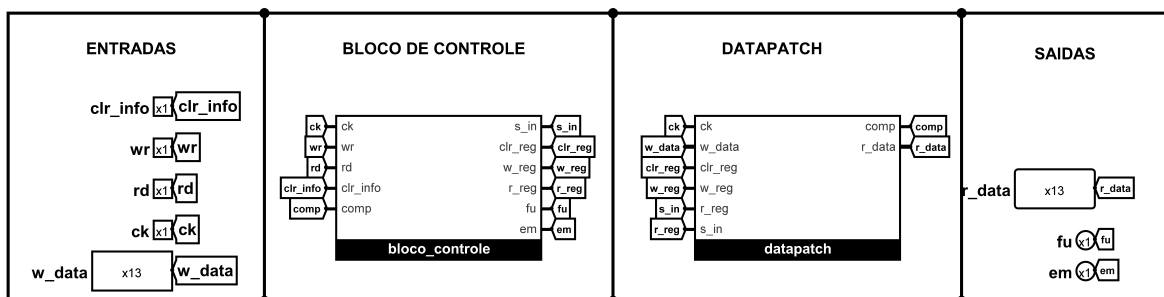


Figura 2.8: Circuito da LIFO

2.2 Projeto da Solução

O projeto completo da solução é baseado na implementação de uma LIFO, responsável pelo armazenamento e recuperação de dados seguindo a lógica de pilha, em que o último elemento inserido é o primeiro a ser retirado. O bloco da LIFO é responsável por toda a lógica da pilha; entretanto, sua saída binária de 13 bits precisa ser convertida em quatro números no formato BCD, permitindo a exibição dos valores em displays de sete segmentos, conforme requisitado no projeto.

Essa conversão pode ser realizada por meio de uma combinação de um circuito BIN para BCD, como o algoritmo Double Dabble, e um circuito de conversão de BCD para os sete segmentos, cuja lógica combinacional é relativamente simples. Neste projeto, foi implementado o Double Dabble de 13 bits e um circuito que converte o BCD nos sete sinais correspondentes de 'a' a 'g' para os displays de sete segmentos.

Ambos os circuitos já foram discutidos e demonstrados em projetos anteriores; seus diagramas são apresentados nas Fig. 2.9 e Fig. 2.10.

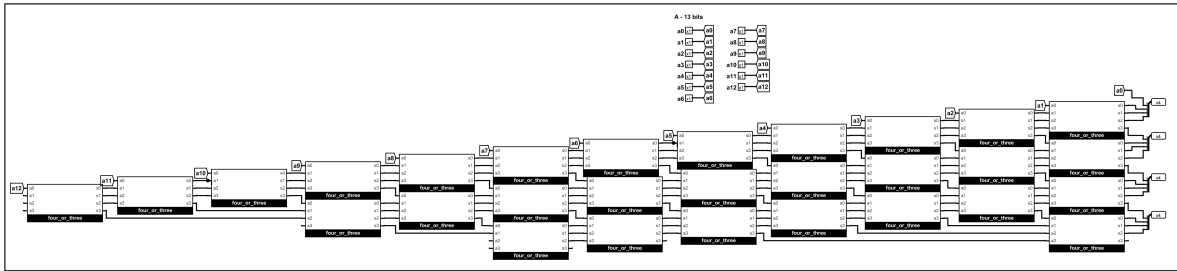


Figura 2.9: Circuito do Double Dabble

Devido ao circuito do Double Dabble ser extenso, não é possível visualizar completamente o encadeamento de todos os elementos; no entanto, ele segue o mesmo princípio dos Double Dabble de menor tamanho. Por outro lado, o circuito de conversão BCD para sete segmentos é mais simples e pode ser visualizado na Fig. 2.10.

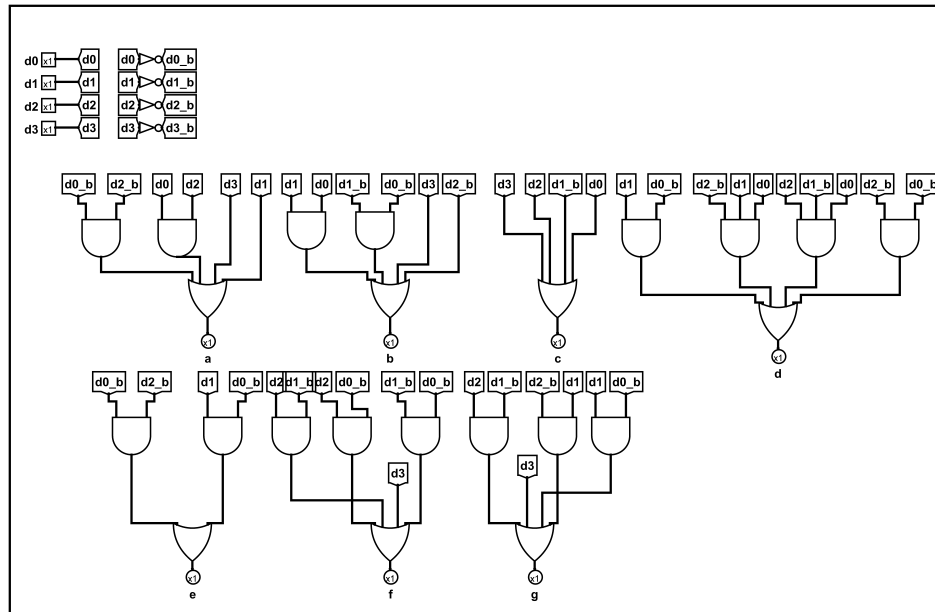


Figura 2.10: Circuito de conversão de BCD para sete segmentos

2.2.1 Montagem Final

A montagem final consiste em juntar as duas grandes partes do circuito, a LIFO e o conversor binário para sete segmentos, seguindo a Fig. 2.11. Para ficar mais organizado foi utilizado a solução de conversor binário para sete segmentos do Logisim, que tem função similar ao discutido no projeto.

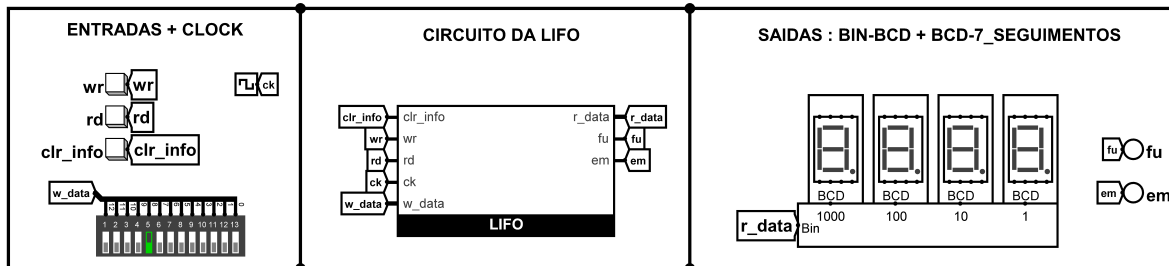


Figura 2.11: Circuito Completo

3 Conclusão

O presente relatório apresentou o desenvolvimento de uma LIFO utilizando a abordagem de projeto RTL, contemplando tanto o `datapath` quanto o bloco de controle. A implementação adotada demonstrou que é possível implementar a lógica da LIFO com apenas um contador, realizando operações de incremento e decremento conforme os estados de escrita e leitura, o que resultou em economia de recursos lógicos sem comprometer a funcionalidade do circuito.

O projeto detalhou a operação da máquina de estados de alto e baixo nível, permitindo um controle preciso sobre as transições entre os estados Inicial, Gravar, Ler e Clear. Além disso, o uso de multiplexadores e lógica combinacional simplificou o design, reduzindo a complexidade do circuito e permitindo uma implementação mais enxuta. O comparador com zero implementado no `datapath` permite detectar quando a pilha estava vazia ou cheia.

Como resultado, o projeto desenvolvido atendeu aos objetivos propostos, fornecendo uma solução funcional e otimizada de uma LIFO, que pode ser aplicada em sistemas de armazenamento temporário, pilhas de processadores ou qualquer aplicação que necessite de acesso em ordem inversa à de escrita.

Referências Bibliográficas

Vahid 2008 VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Artmed, 2008. 560p; 25 cm. ISBN 978-85-7780-190-9.

A Relato semanal

A.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder	Samuel de Lira Moraes	Funções: Líder. Relatório partes : Fundamentação teórica.
Redator	Rinaldo Tavares da Silva Filho	Função: Relatório partes: Máquina de estado, Projeto RTL, Datapath, Bloco de controle.
Debatedor	Christian Cauã Batista da Silva	Funções: Relatório.

A.2 Defina o problema

Projeto de um maquina de troco.

A.3 Pontos-chaves

Projeto RTL, maquina de estados de alto nível;

A.4 Questões de pesquisa

Projetos RTL, maquina de estados de alto nível;

A.5 Planejamento da pesquisa

([Vahid 2008](#)) apenas.

B Anexos

Tabela B.1: Tabela da Verdade logica de controle

l_state	wr	rd	clr_lifo	$comp$	$state$	S_in	clr_reg	w_reg	r_reg	fu	em
0	0	0	0	x	0	0	0	0	0	0	0
0	0	x	x	1	x	1	1	0	0	0	0
0	0	0	1	0	x	1	0	1	0	0	0
0	0	1	x	0	x	0	1	0	0	0	0
0	1	x	x	x	0	0	0	0	0	0	0
0	1	x	x	x	1	0	0	0	0	1	0
1	0	x	x	x	0	0	0	0	0	0	0
1	x	x	x	x	1	0	0	0	0	0	1
1	1	x	x	x	0	0	0	0	0	0	1

Tabela B.2: Tabela da Verdade Decoder

$cmd[3..0]$	$enable$	$e1$	$e2$	$e3$	$e4$	$e5$	$e6$	$e7$	$e8$	$e9$	$e10$	$e11$	$e12$	$e13$	$e14$	$e15$	$e16$
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0

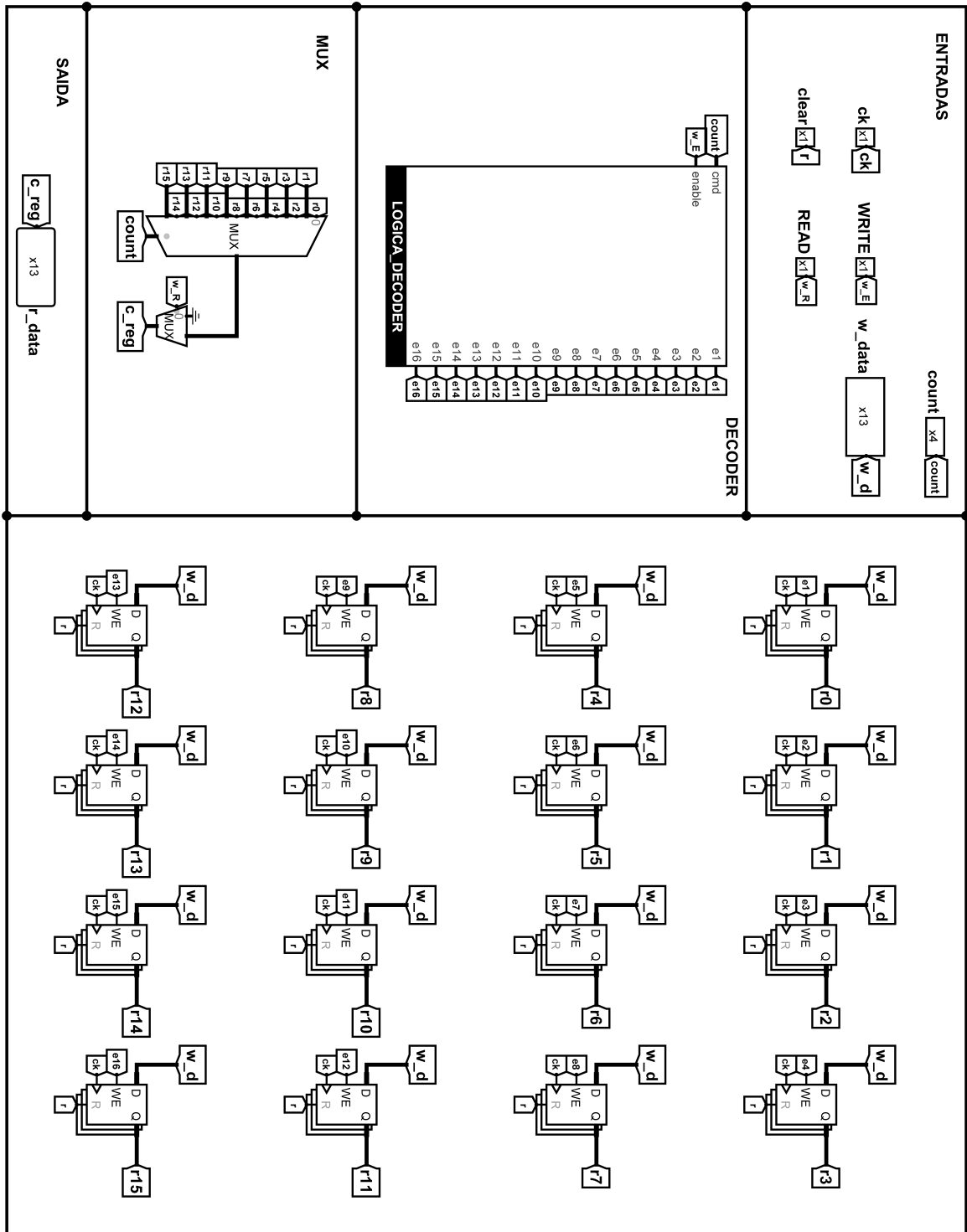


Figura B.1: Estrutura da memória utilizada no projeto LIFO

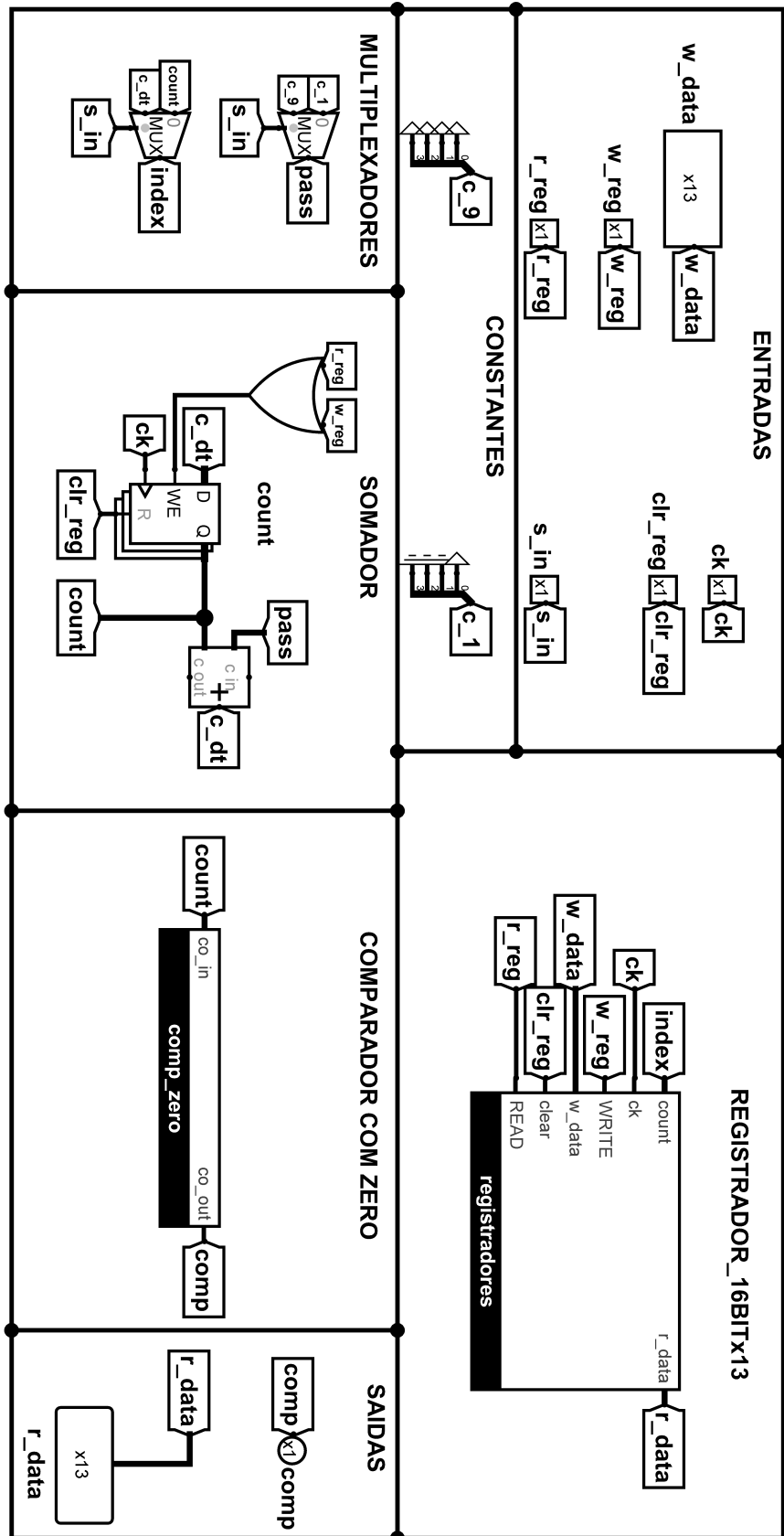


Figura B.2: Datapath do projeto LIFO

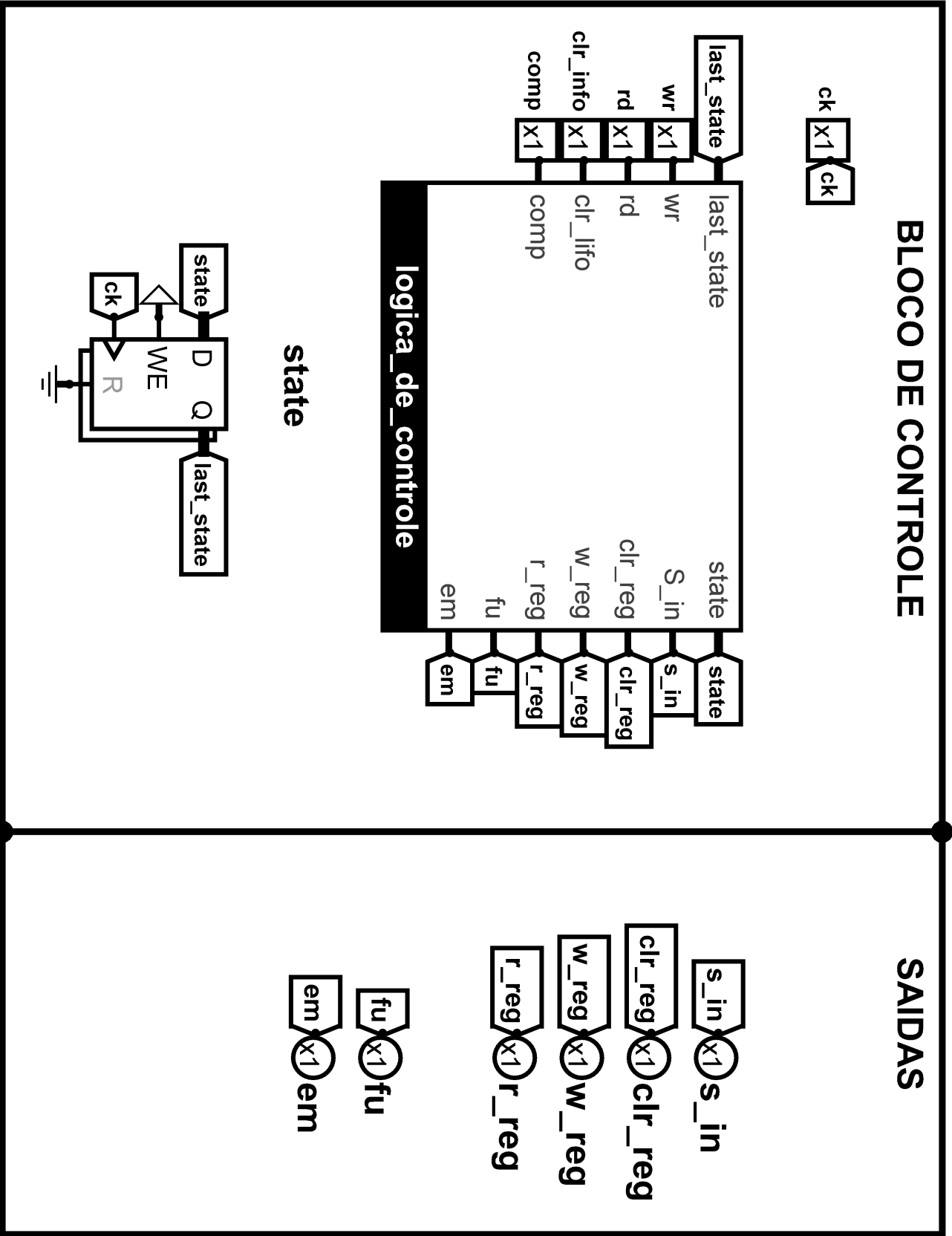


Figura B.3: Bloco de controle responsável pelas operações da LIFO

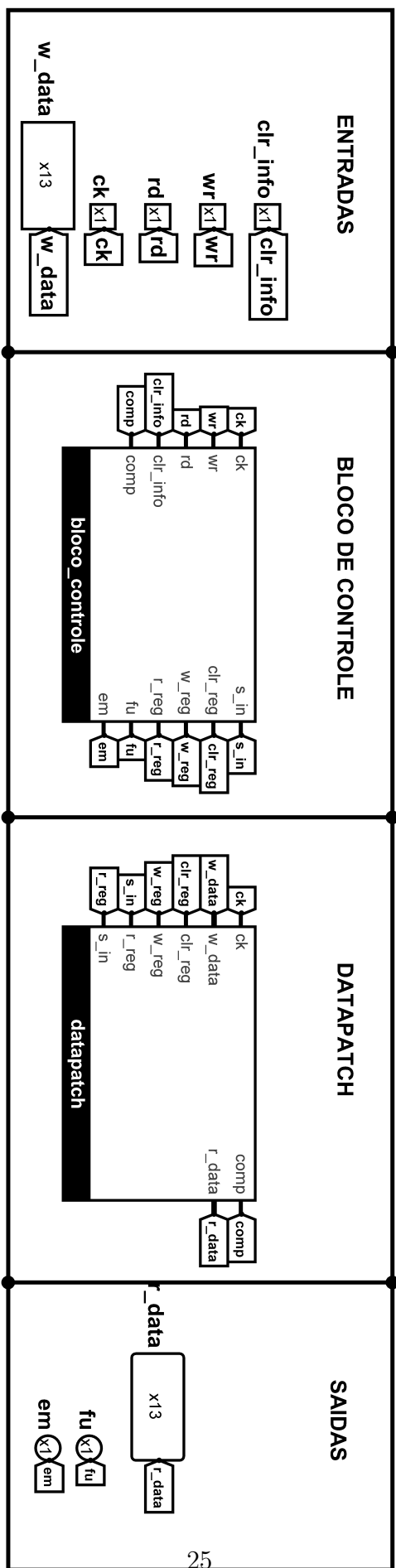


Figura B.4: Diagrama geral do circuito LIFO implementado

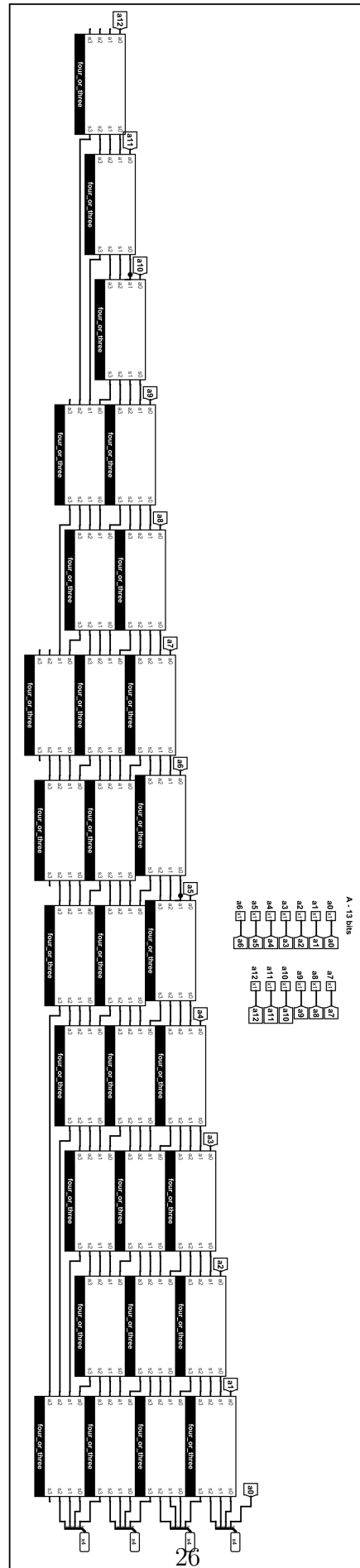


Figura B.5: Conversor de saída binária de 13 bits para formato BCD